Sam Duddy
CS5200 Final Project – README
Potluck App Database and Backend

1. **Creating and Running Project**

   This project uses PostgreSQL as a database and node.js as the backend framework, which can be downloaded here: https://nodejs.org/en/download

   First, the database will need to be built in PostgreSQL. This can be done by creating a new database (named 'potluckApp') and executing the 'duddy_final_project.sql' script to build the tables, relationships, and populate initial data.

   Once node.js has been downloaded as well as the application files, you will need to initialize the project as NPM (this is the node.js package library). To do this, use the following command line command once you are within the application directory:

   $ npm init

   After the project has been initialized, several of the NPM modules will need to be installed, as they are used by the application. The following commands will install the necessary modules:

   $ npm i date-and-time
   $ npm i dotenv
   $ npm i pg
   $ npm i validator
   $ npm i yargs@12.0.2

   Also verify that the information in the development.env folder matches the credentials for the PostgreSQL server that you are using. Now, the database and backend should be connected and ready for use.

2. **Technical Specifications**

   a. **Overview/Problem Description**

      For my final project, I built a database with backend command-line-based functionality for a potluck scheduling app. The idea was to enable a user to create an account, schedule potluck events, send invitations, respond to

invitations, as well as post messages to events and comment on posts from other users.

For the database I used PostgreSQL in combination with DBeaver, and for the backend I used node.js. The application functionality is currently based on command line arguments that represent the actions that a user would be able to take from an eventual frontend.

b. Background/Interest

The project interests me because my roommates and I currently host potlucks at our apartment. We value the sense of community and connection that they foster but often struggle to coordinate dishes and guests. To better facilitate this, a potluck scheduling app with the ability for a guest to say what dish they plan to bring seemed like a good idea. It is also tied into the ability of technology to actually help people come together in the real world, which is inspiring to me.

c. How it Works

The backend framework I used was node.js. Each of the tables in my database has their own corresponding .js file in the backend where methods are defined allowing CRUD operations to be performed. An index.js file is used to create a connection with the database using the 'pg' and 'dotenv' npm modules. Environment variables are stored in the development.env file so that they can be changed more easily.

The functionality of the application is done through the app.js file, where each of the disparate methods corresponding to different CRUD operations on the various database tables are unified into unique command line arguments that a user can take advantage of. This is done through the 'yargs' npm module that allows for clean and organized command line argument definitions. At the top of app.js file, each of the corresponding entity files are linked using the built-in 'require()' method so that they can be called upon execution of command line arguments.

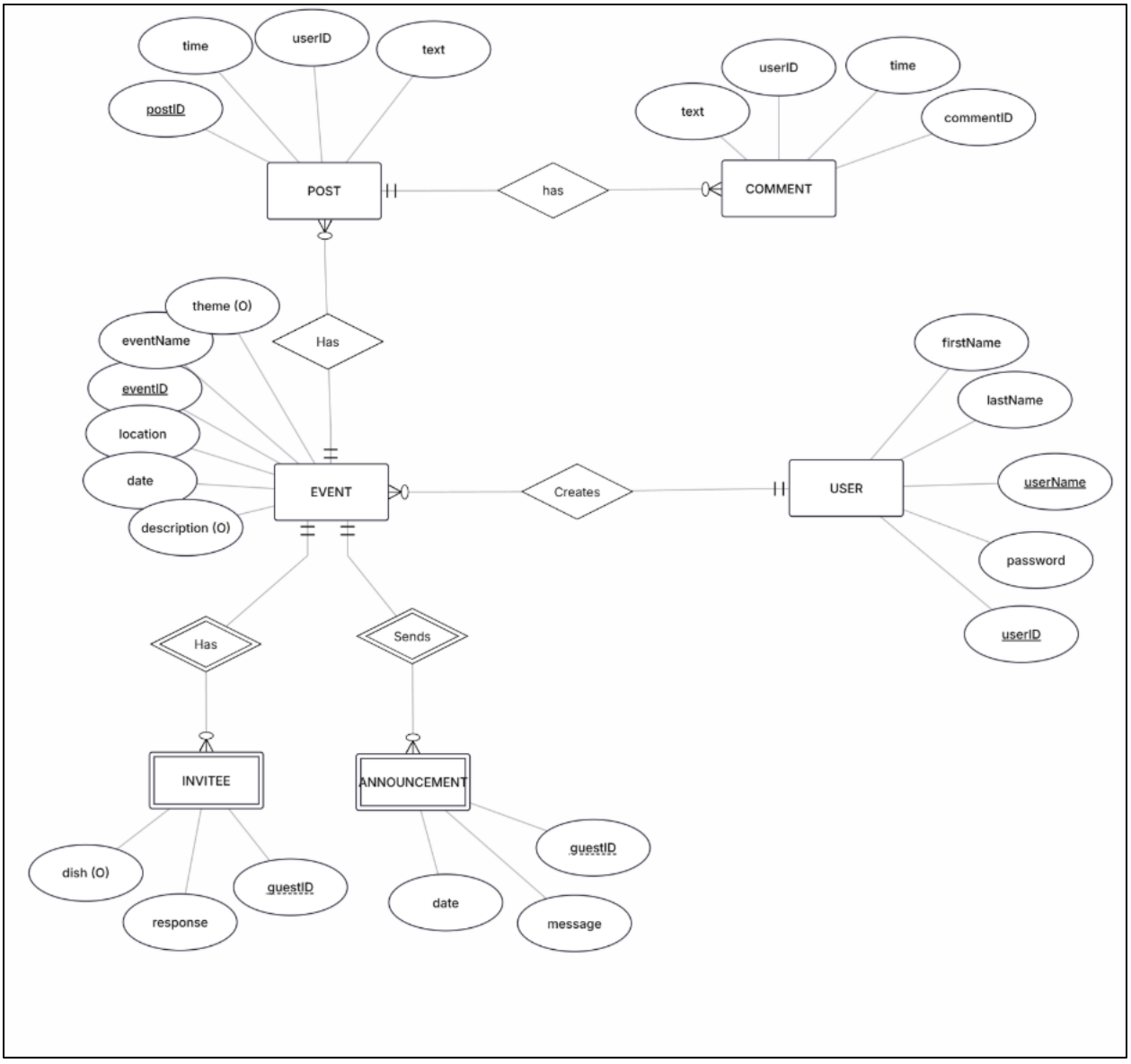
d. Future Goals

As it stands now, the user's functionality is based only on command line arguments that allow for performing CRUD operations. The obvious future goal would be to build a proper frontend that could exist as a standalone web

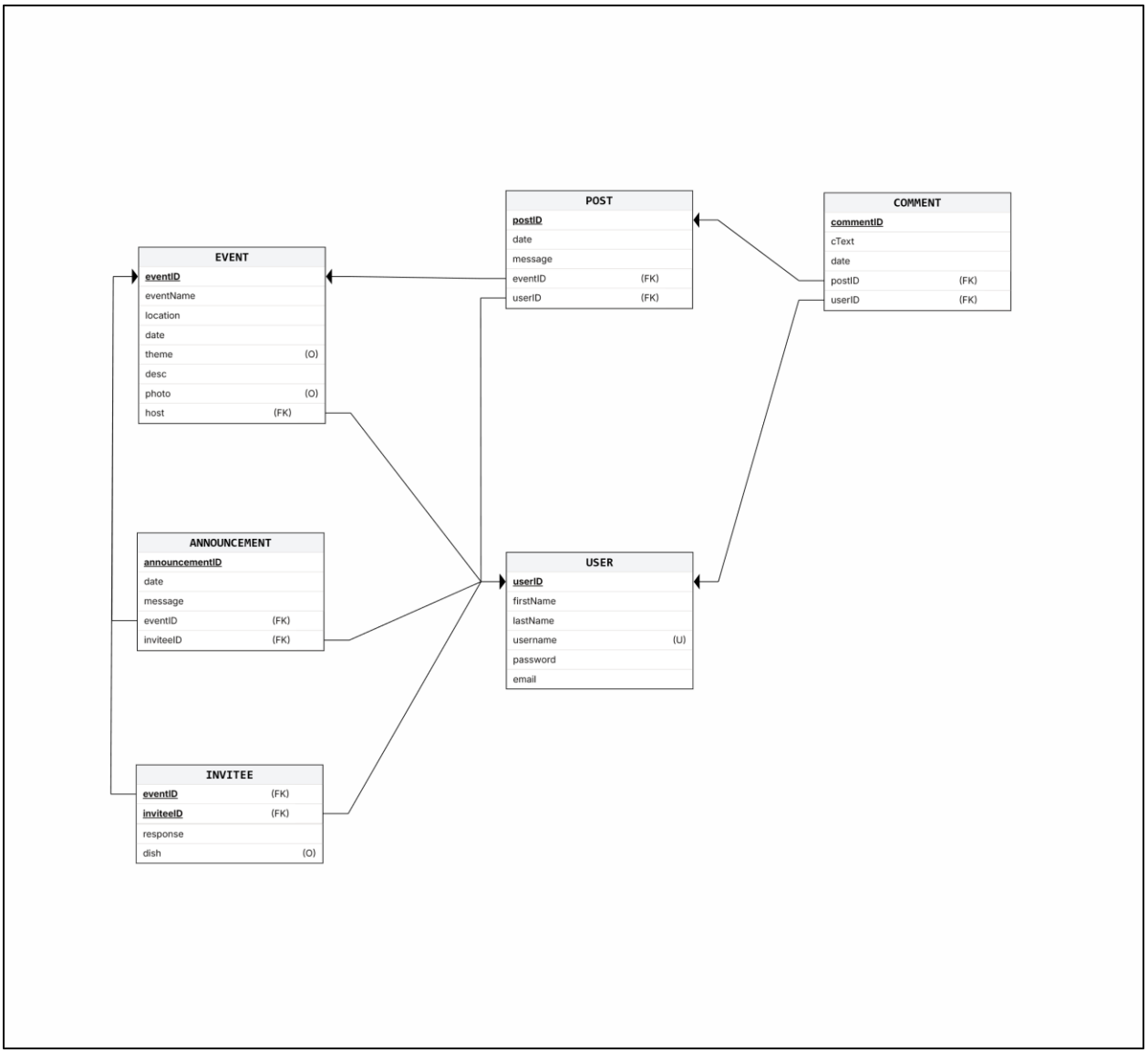
application. From here, a user could interact with the application through a GUI that would allow for even more streamlined scheduling of events and visualizing past and upcoming events.

3. Conceptual Design

ER Diagram:



Relational Schema:



4. User Flow

As mentioned above, a user will interact with the application through command line arguments to perform basic CRUD operations. All command line arguments, along with the parameters they require, are listed below and are grouped by which of the database tables they primarily interact with:

Relation (Entity)	CRUD Operation	Command	Params
APPUSER	create	add_user	f_name, l_name, username, password, email
APPUSER	delete	delete_user	username
APPUSER	read	get_users	None

APPUSER	update	update_username	userID, username
EVENT	create	create_event	name, address, date, theme (o), description (o), photo (o), host
EVENT	delete	delete_event	name
EVENT	read	get_host_events	host userID number
EVENT	read	get_upcoming_events	userID number
EVENT	read	get_past_events	userID number
EVENT	update	update_event_date	eventID, date
INVITEE	create	invite	eventID, userID
INVITEE	delete	uninvite	eventID, userID
INVITEE	update	rsvp	eventID, userID, response, dish,
INVITEE	read	get_responses	eventID
POST	create	post	eventID, userID, text
POST	delete	delete_post	postID
POST	read	get_posts	eventID
POST	update	update_post	postID, message
COMMENT	create	comment	postID, text, userID
COMMENT	read	get_comment	postID
COMMENT	update	update_comment	commentID, comment
COMMENT	delete	delete_comment	commentID
ANNOUNC-EMENT	create	send_announcement	eventID, userID, text
ANNOUNC-EMENT	read	get_announcement	userID
ANNOUNC-EMENT	update	update_announcement	announcementID, message
ANNOUNC-EMENT	delete	delete_announcement	announcementID

Note that to run a command line argument using node.js, the following syntax is required:

$ node app.js *command* --param1='value' --param2='value'

Where 'node' is the command used to run any node.js program, 'app.js' is the name of the main application file where the command line arguments are defined and their respective functions referenced, '*command*' indicates which of the above commands is intended to be used, and finally – each of the parameters is defined using a double dash, '--', followed by the parameter name, an equals sign, and the

parameter value in quotes. For example, the way to execute an add_user command would be:

```
node app.js create_event --name='January Potluck' --address='74 Cumberland Ave' --date='01/17/2026' --host='1'
```

5. **Lessons Learned**
    a. *Technical Expertise Gained:*
    This project greatly enhanced my ability to understand the role that databases play in applications, the importance of proper system design, and how to build a backend using a new framework, Node.js. Along with this, I feel as though I have a clearer understanding of the software development process and the complexity that is unavoidable with large interrelated systems. In my exploration of Node.js, it was cool to see how much overlap from CS 5004 (Object Oriented Design) there was. I found myself well prepared to apply some of the same foundational knowledge to a different framework.
    b. *Insights, Time Management, Data Domain:*
    One of the major insights gained from this project is the importance of the initial design of a database. Although challenging, having a well-thought-out ER diagram seems to make all of rest of the implementation (although much more time consuming) much more streamlined. I have also seen that thinking about an applications intended functionality and then attempting to distill it down into discrete entities and relationships is a muscle that takes some time to develop. I think that one of the most rewarding parts of the project for me was finally arriving at a satisfying ERD that I felt was both organized, logical, and functional.
    c. *Realized or Contemplated Alternative Design*
    One of the main aspects of the design that I contemplated as an alternative was how to incorporate the entity of INVITEE. Given that this must also represent a USER (already an entity) but not a host of an event, this was one part of the database design that I went back and forth on. The alternative that I contemplated was instead of implementing this as its own relation, to use the USER relation with additional attributes to capture properties of an INVITEE. Yet, as I discovered, this made a lot less sense and was more difficult to represent visually.

I also contemplated using Python as the backend framework but given that much of my programming experience has been in Python, I was more interested in learning a new language, so I decided to go with Node.js. To help me, I used a Udemy course "The Complete Node.js Developer Course" that was greatly helpful for me to learn the basics enough to apply them to this final project implementation.

d. *Code not working: NA*

e. *Future Work*

   i. *Planned Uses of the Database*

   I would eventually love to use this database within my friend group at our own potlucks to better track guests and dishes. It would also be very cool to host it as a stand alone web application that would allow anyone to use it to schedule their own potlucks easily by simply signing up as a new user.

   ii. *Potential areas for Added Functionality*

   There are many areas that I would like to build out the functionality to be more nuanced and extensive. The first and most important thing would be to develop a frontend application where a user could interact with a GUI rather than a command line. Additionally, more granularity and flexibility allowing a user to perform UPDATE operations on different aspects of their profiles and events would be great. I'd also love to implement a direct messaging feature where users could communicate with one another outside of the context of an event.

   iii. NA